

RLS Timing Side-Channel Attack: Hospital Patient Data Example

Scenario Setup

Database: Hospital patient records stored in PostgreSQL with Row-Level Security

Table: `patients`

PK	Patient_ID	Name	Age	Salary	Site_ID
1	5001	John Smith	45	120k	1
2	5002	Alice Johnson	32	95k	2
3	5003	Bob Williams	67	150k	1
4	5004	Carol White	28	75k	3
5	5005	Dave Brown	55	130k	2

RLS Policy:

- Doctor from Site 1 can only view patients from Site 1
- Doctor from Site 2 can only view patients from Site 2
- Doctor from Site 3 can only view patients from Site 3

Attacker: A doctor at Site 1 who wants to learn about patients at Site 2 (unauthorized access)

Step 1: Understanding the Timing Difference

Before Attack (Baseline)

The attacker first runs some queries to understand the timing patterns:

Query A - Non-existent patient (exists nowhere in DB):

```
sql
SELECT * FROM patients WHERE patient_id = 9999;
```

- Index lookup: ~350 microseconds
- Found: NO
- RLS policy evaluation: NOT applied (0 rows match)
- Total time: ~350 microseconds

Query B - Authorized patient (belongs to Site 1):

```
sql
```

```
SELECT * FROM patients WHERE patient_id = 5001;
```

- Index lookup: ~350 microseconds
- Found: YES (1 row)
- RLS policy evaluation: Applies policy filter to 1 row
- Policy execution: ~130 microseconds (checking if doctor's site matches)
- Total time: ~480 microseconds

Query C - Unauthorized patient (belongs to Site 2):

sql

```
SELECT * FROM patients WHERE patient_id = 5002;
```

- Index lookup: ~350 microseconds
- Found: YES (1 row exists)
- RLS policy evaluation: Applies policy filter to 1 row
- Policy execution: ~130 microseconds (checking if doctor's site matches)
- Total time: ~480 microseconds
- Result returned: NO DATA (filtered out by RLS policy)

Key Observation

```
Query A (Non-existent):    350 microseconds
Query B (Authorized):      480 microseconds
Query C (Unauthorized):    480 microseconds
```

The Problem: Query B and Query C take the SAME time, even though Query C is for unauthorized data!

This timing reveals: **The patient record exists in the database** (even though the data is filtered out).

Step 2: Breaking Existence Privacy

Attacker's Goal

Figure out if patient ID 5002 (Alice Johnson from Site 2) exists in the database.

Attack Procedure

The attacker is a doctor at Site 1. They run these queries:

```
sql

-- Query a non-existent patient (as reference point)
SELECT * FROM patients WHERE patient_id = 9999;
```

Measure time 10 times, take minimum: ~**350 microseconds**

```
sql

-- Query a patient they KNOW exists and CAN see (Site 1)
SELECT * FROM patients WHERE patient_id = 5001;
```

Measure time 10 times, take minimum: ~**480 microseconds**

```
Threshold = (350 + 480) / 2 = 415 microseconds
```

Step 2B: Query the Target (Unauthorized)

```
sql

-- Query patient from Site 2 (attacker shouldn't know they exist)
SELECT * FROM patients WHERE patient_id = 5002;
```

Measure time 10 times, take minimum: ~**475 microseconds**

Result of Attack

```
Query Time: 475 microseconds
Is 475 > 415 threshold? YES
```

Conclusion: Patient 5002 EXISTS in the database (even though no data was returned).

The attacker now knows: **"There is a person with patient ID 5002 in the hospital database, even though I'm not authorized to see them."**

Step 3: Breaking Cardinality Privacy (Advanced)

Now imagine the attacker wants to learn **the AGE** of an unauthorized patient.

Attack: Binary Search on Age

Target: Find the age of patient 5002 (Alice Johnson)

Query with age predicate:

sql

```
SELECT * FROM patients
WHERE patient_id = 5002 AND age > 25;
```

The query optimizer will:

1. Evaluate the index on (patient_id) → Find 1 row
2. Evaluate age filter (age > 25) → Returns 1 row (matches)
3. Evaluate RLS policy → Filters out (unauthorized)
4. Total time: ~475 microseconds

sql

```
SELECT * FROM patients
WHERE patient_id = 5002 AND age > 50;
```

The query optimizer will:

1. Evaluate the index on (patient_id) → Find 1 row
2. Evaluate age filter (age > 50) → Returns 0 rows (doesn't match)
3. Evaluate RLS policy → Applied to 0 rows
4. Total time: ~350 microseconds

Binary Search Breakdown

The attacker now knows:

- Time for (age > 25): ~475 microseconds (patient's age > 25)
- Time for (age > 50): ~350 microseconds (patient's age ≤ 50)

Using binary search:

Age range: 1-100

Query: age > 50? → 350 μs (NO) → Range: 1-50

Query: age > 25? → 475 μs (YES) → Range: 26-50

Query: age > 38? → 475 μs (YES) → Range: 39-50

Query: age > 44? → 350 μs (NO) → Range: 39-44

Query: age > 42? → 475 μs (YES) → Range: 43-44

Query: age > 43? → 350 μs (NO) → Age = 43? (close enough)

Actually Actual age: 32 (within a few years)

Step 4: Extracting Multiple Attributes (Compounding Attack)

Once the attacker knows Patient 5002 exists and their approximate age, they can search for other attributes:

Finding Salary

Repeat the same process with salary predicates:

```
sql

SELECT * FROM patients
WHERE patient_id = 5002 AND salary > 50000; → 475 μs (YES)

SELECT * FROM patients
WHERE patient_id = 5002 AND salary > 100000; → 350 μs (NO)

SELECT * FROM patients
WHERE patient_id = 5002 AND salary > 75000; → 350 μs (NO)

SELECT * FROM patients
WHERE patient_id = 5002 AND salary > 60000; → 475 μs (YES)
```

Binary search reveals: Salary is between \$60k-\$75k (actual: \$95k, but attacker gets close)

Step 5: Complete Attack Timeline

Day 1: Initial Reconnaissance

```
10:00 AM - Attacker measures baseline query times
10:05 AM - Attacker establishes threshold: 415 microseconds
10:10 AM - First attack query for patient 5002 (site 2)
Result: FOUND - Patient exists
```

Day 1-2: Attribute Extraction via Binary Search

```
10:15 AM - Search age range → Found: ~32 years old
10:45 AM - Search salary range → Found: ~$75k-$95k
11:15 AM - Search department via text search
...
```

Day 3: Privacy Breach Complete

Patient ID: 5002 (confirmed)
Name: Unknown (still cannot see this)
Age: ~32 years old ✓ (extracted)
Salary: ~\$90k ✓ (extracted)
Site: 2 (confirmed via existence check)

What's still hidden by RLS: The name "Alice Johnson" cannot be returned by any query

Why This Works: The Timing Formula

The key insight is this mathematical relationship:

Total Query Time = Query Execution Time + RLS Policy Execution Time

Query Execution Time = ~350 microseconds (constant for indexed lookups)

RLS Policy Execution Time = Number of Matching Rows × Per-Row Policy Cost

For 0 matching rows: $350 + (0 \times 130) = 350 \text{ } \mu\text{s}$
For 1 matching row: $350 + (1 \times 130) = 480 \text{ } \mu\text{s}$
For 10 matching rows: $350 + (10 \times 130) = 1,650 \text{ } \mu\text{s}$

The timing leaks the cardinality (number of rows matching the user's query) **BEFORE RLS filtering.**

Real Numbers from the Paper

From the paper's actual experiments on AWS:

Measurement	PostgreSQL	SQL Server
Non-existent key:	377.25 μs	386.01 μs
Authorized key:	880.87 μs	412.80 μs
Unauthorized key:	889.7 μs	404.49 μs
Difference:	512.45 μs	18.48 μs

PostgreSQL: Clear separation of 512 microseconds **SQL Server:** Much smaller gap due to optimization, but still detectable

Attack success rate: **99.77%** with just 10 queries

Visual Timeline of Attack

```
| Normal Doctor (Site 1) querying |  
| SELECT * FROM patients WHERE id = 5001 |  
| (Authorized - site 1 patient) |  
|  
| Time: 480 µs |  
| Result: Shows data ✓ |
```

↓↓↓

```
| Attacker (Site 1) with timing attack |  
| SELECT * FROM patients WHERE id = 5002 |  
| (Unauthorized - site 2 patient) |  
|  
| Time: 480 µs ← SAME TIMING! |  
| Result: No data shown × |  
| But: Can infer patient exists |
```

Attacker's Conclusion:

"Query took same time (480 µs) as authorized queries,
not the 350 µs of non-existent queries.
Therefore: Patient 5002 EXISTS in database!"

What the Mitigation Does

Without Mitigation (Vulnerable)

```
SELECT WHERE id=9999 (non-existent): 350 µs  
SELECT WHERE id=5002 (unauthorized): 480 µs ← Information leaked!
```

With Mitigation (Proposed Solution)

```
SELECT WHERE id=9999 (non-existent): 480 µs ← Fake RLS execution  
SELECT WHERE id=5002 (unauthorized): 480 µs ← Real RLS execution  
          ^ Always the same!
```

The mitigation forces the database to:

- Always evaluate the RLS policy
 - Even when no rows match the user's query
 - Making timing completely uniform
-

Key Takeaway

The attacker never sees the actual data, but can extract sensitive information through timing measurement alone.

This is why it's called a "side-channel" attack—it attacks through a side effect of the system (execution time), not through the main mechanism (returning data).

The attack demonstrates that:

1. ✓ RLS successfully prevents unauthorized data from being returned
2. ✗ But cannot prevent information leakage through execution timing
3. ✗ Attacker can discover existence and extract attributes through binary search
4. ✓ The proposed fix makes execution time constant (data-oblivious)